MICROSOFT SENTINEL
CLOUD SOC LAB

Personal Lab Project | Portfolio Evidence

Purpose: This document describes a beginner cloud security monitoring lab built on Microsoft Azure. Azure Activity logs are collected into a Log Analytics workspace, analysed in Microsoft Sentinel with Kusto Query Language (KQL), and monitored by a scheduled detection rule for repeated failed operations. All subscription, tenant, workspace and account identifiers have been removed from the screenshots.

1. Project objective

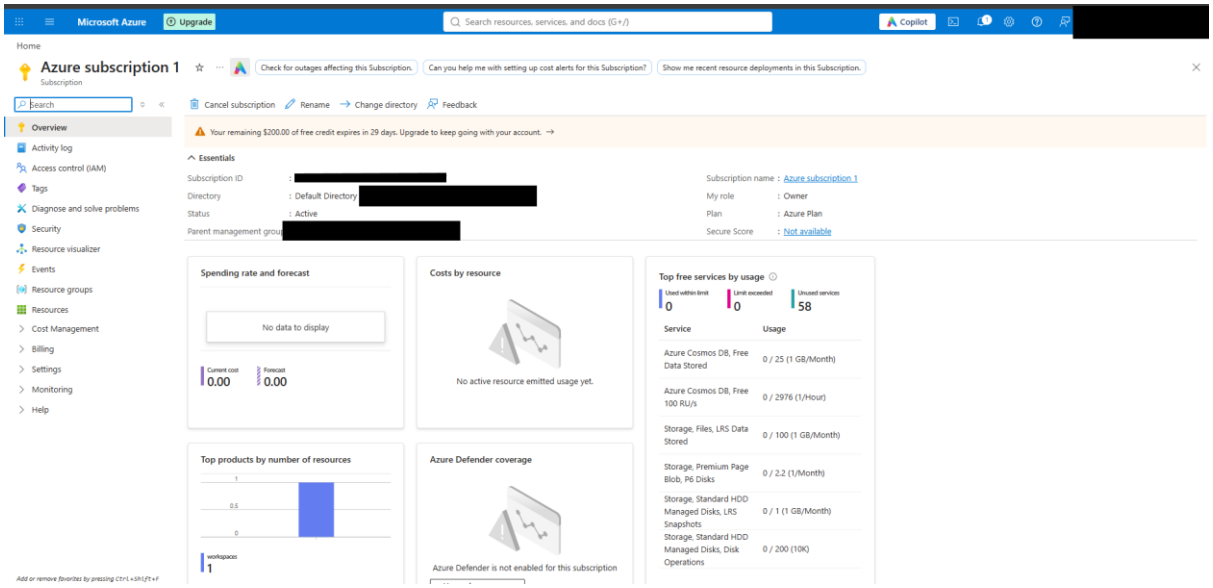
The objective of this lab was to build a small but complete cloud monitoring pipeline and practise the basic workflow of a SOC analyst working in the cloud. Specifically, the lab aimed to:

- Collect Azure Activity logs from an Azure subscription into a Log Analytics workspace.
- Use KQL to search for failed operations and read who did what, when and where.
- Configure a simple scheduled analytics rule that flags three or more failed operations by the same caller within a 15-minute window.
- Practise reviewing the resulting alerts, or documenting clearly why no alert was generated.

2. Architecture

Data flow: Azure activity → Log Analytics workspace → Microsoft Sentinel → SOC analyst review.

Component	Role in this lab
Azure Activity logs	Source events: control-plane operations on the subscription such as create, update, delete, deployments, role assignments and policy assignments.
Log Analytics workspace	Stores the collected data in searchable tables. The table used here is AzureActivity.
Microsoft Sentinel	Cloud-native SIEM that provides data connectors, analytics rules and incident workflows.
SOC analyst (me)	Runs KQL queries, reviews alerts, investigates context and documents findings.

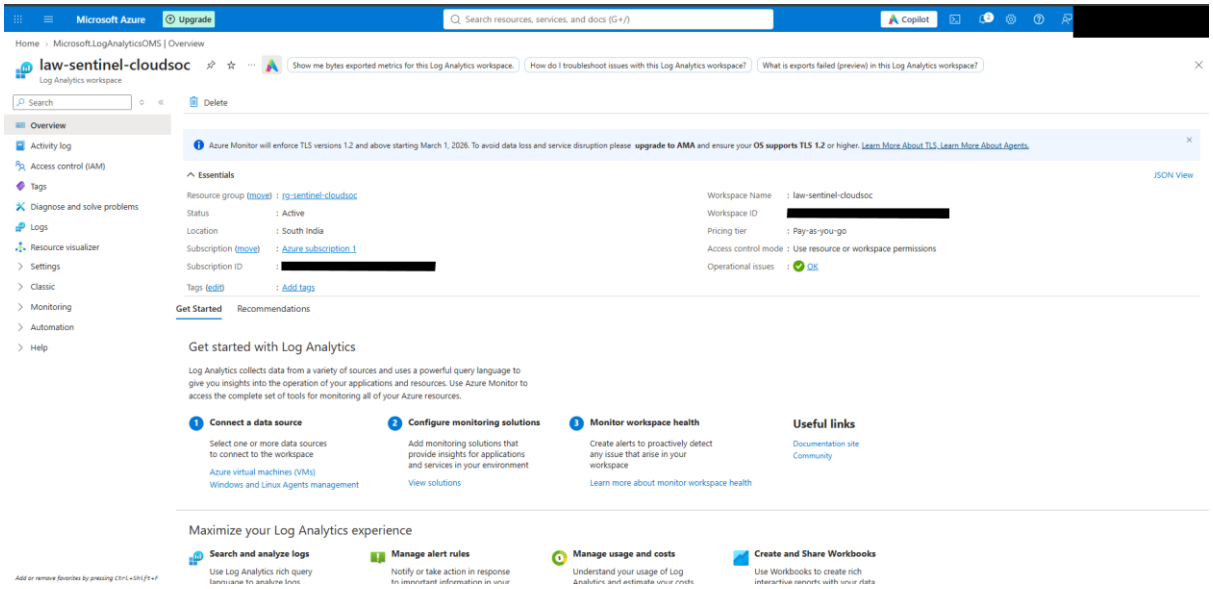


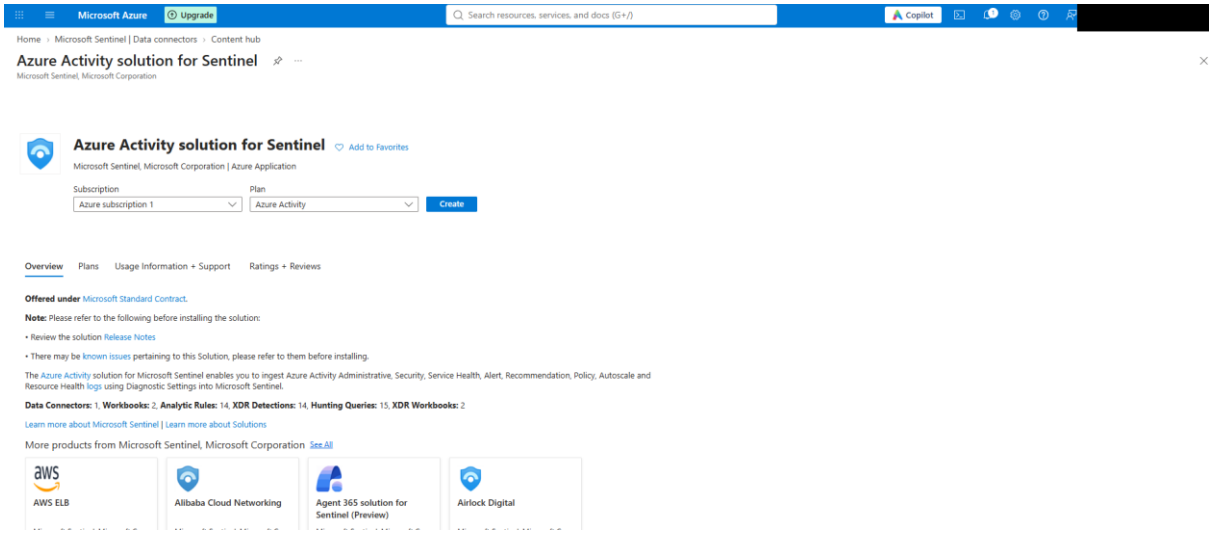
The architecture diagram gives the overview; the sections below show evidence from the actual Azure portal for each step.

3. Environment setup

The lab was built in a personal Azure subscription using free credit, with the Owner role. Cost analysis showed no spend at the time of capture. The following resources were created:

Item	Value
Resource group	rg-sentinel-cloudsoc
Log Analytics workspace	law-sentinel-cloudsoc
Region	South India
Workspace pricing tier	Pay-as-you-go
Access control mode	Use resource or workspace permissions
Microsoft Sentinel	Enabled on law-sentinel-cloudsoc

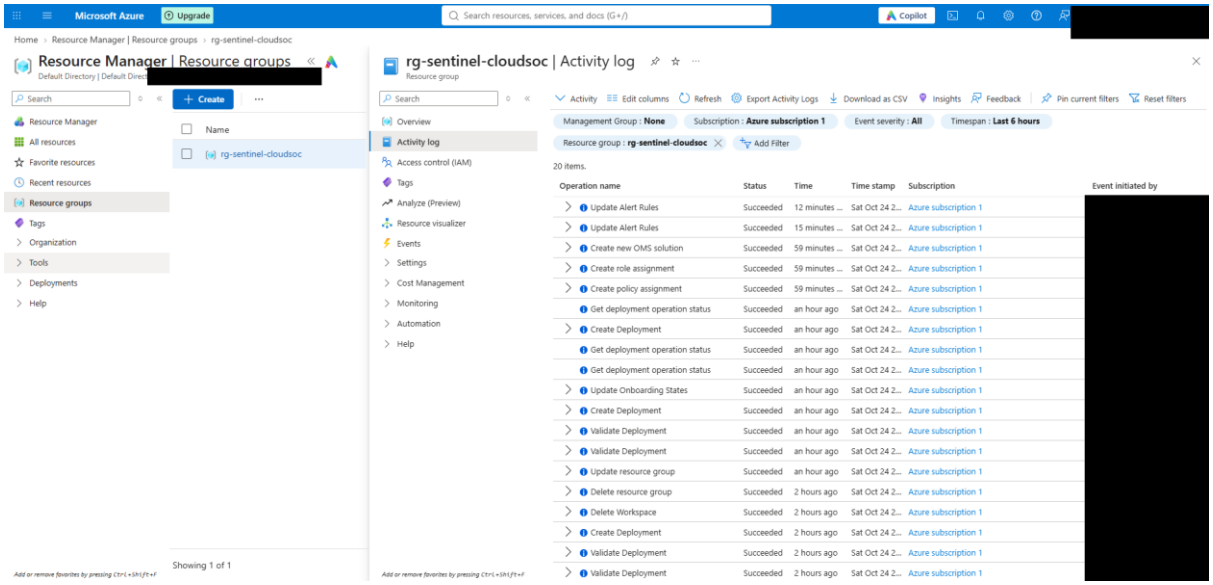




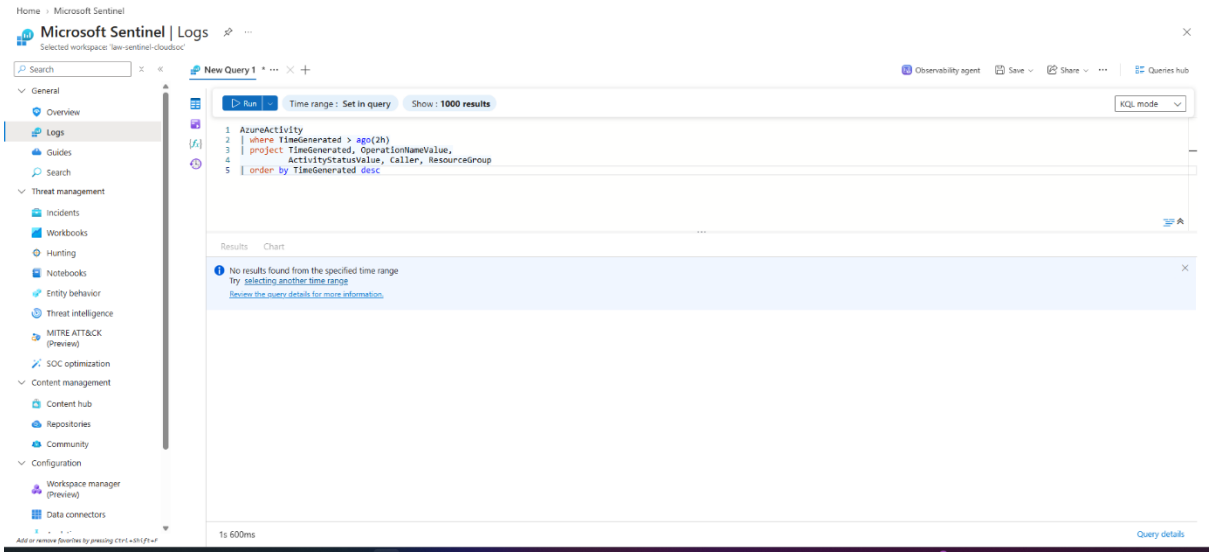
The workspace overview shows an active status with no operational issues. Subscription ID, workspace ID and tenant details are not shown.

4. Azure Activity data connection

From Microsoft Sentinel → Content hub, I used the Azure Activity solution for Sentinel. The solution provides the Azure Activity data connector together with related workbooks, analytics rule templates and hunting queries. The connector sends subscription activity to the selected Log Analytics workspace.



The Activity log of the resource group also records the setup operations as they happened: deployments, creation of the OMS solution, a role assignment, a policy assignment, onboarding-state updates and alert-rule updates. This gives a timeline of the build and shows the type of events that the AzureActivity table is designed to hold.

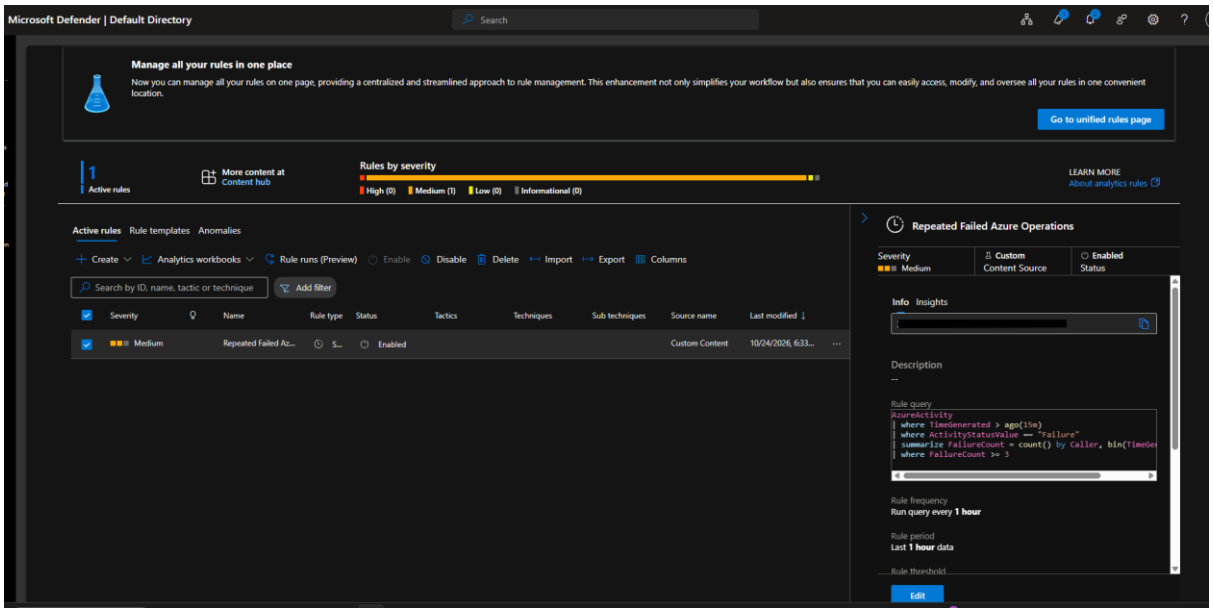


5. Log collection validation

To check that data was reaching the workspace, I ran the following query in Microsoft Sentinel → Logs:

```
AzureActivity
| where TimeGenerated > ago(2h)
| project TimeGenerated, OperationNameValue,
          ActivityStatusValue, Caller, ResourceGroup
| order by TimeGenerated desc
```

Result: the query returned “No results found from the specified time range” for the last two hours.



Interpretation: the connector had only just been configured, and newly connected sources can take time before records appear in the workspace. An empty result at that moment does not prove the connector is broken, but it also does not prove ingestion works. Validation is only complete once the table returns rows, so the next step is to widen the time range (for example ago(24h) or ago(7d)) and re-run the query.

Author note: *re-run this query once data has arrived, replace Figure 6 with a screenshot that shows rows, and update the result above.*

6. KQL investigation – failed operations

Once records are present, the following query filters recent Azure activity to failed operations and shows the fields an analyst needs first: time, caller, operation, resource group and status.

```
AzureActivity
| where TimeGenerated > ago(24h)
| where ActivityStatusValue =~ "Failure"
| project TimeGenerated, Caller, OperationNameValue,
          ResourceGroup, ActivityStatusValue
| order by TimeGenerated desc
```

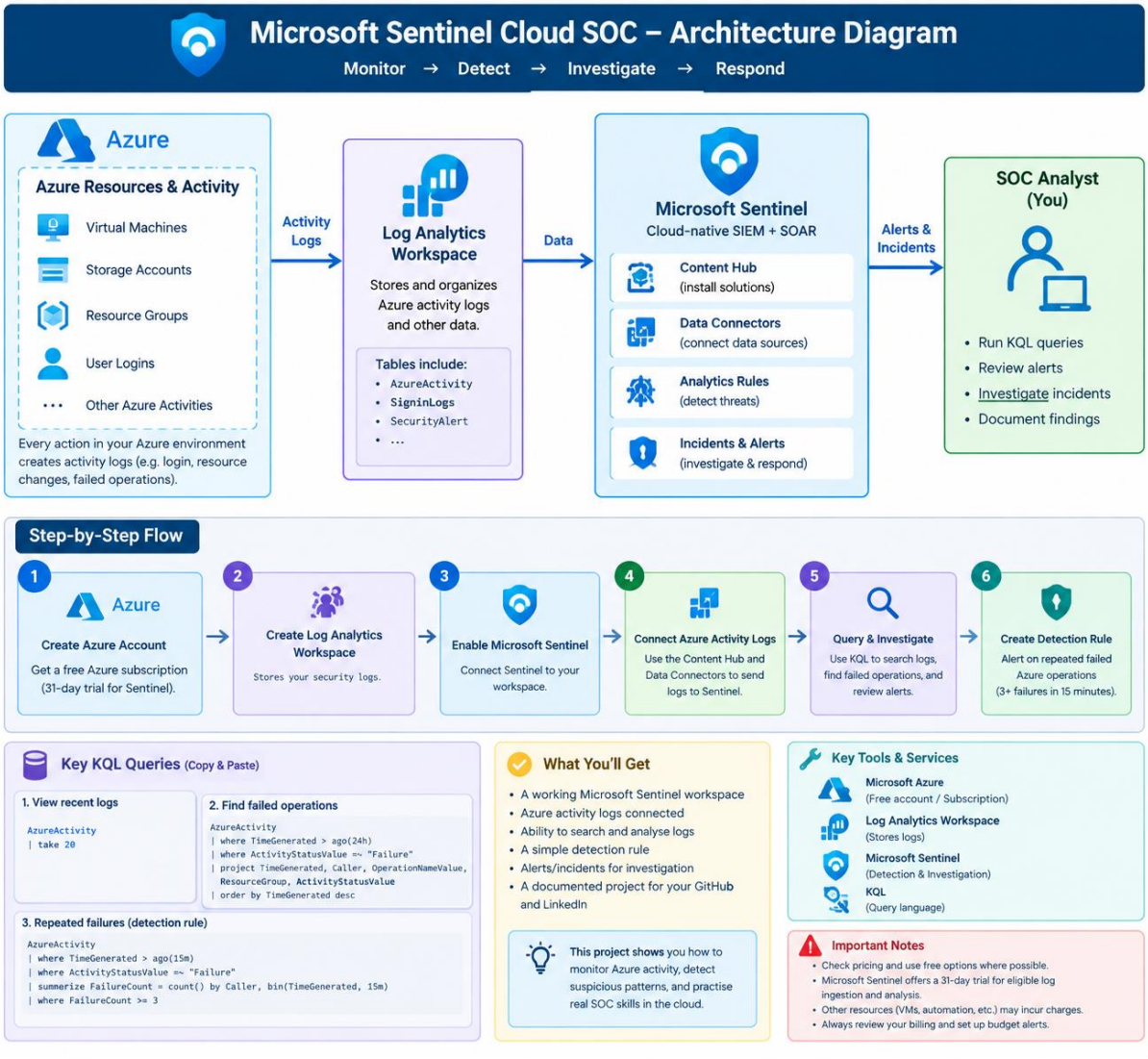
A failure is not automatically malicious. Typical benign causes include missing permissions, a typo in a resource name, a policy that blocks a deployment, or a quota limit. The analyst should look at context: who the caller is, whether it is a person or a service principal, whether the pattern is new, and which resource was targeted.

7. Detection rule

I created a scheduled analytics rule named Repeated Failed Azure Operations to identify three or more failed operations by the same caller within a 15-minute bucket.

Setting	Value
Rule name	Repeated Failed Azure Operations
Rule type	Scheduled
Severity	Medium
Status	Enabled
Source	Custom content
Query frequency	Every 1 hour
Lookback period	Last 1 hour
Threshold logic	3 or more failures per caller per 15-minute bin (enforced in the query)

```
AzureActivity
| where TimeGenerated > ago(15m)
| where ActivityStatusValue =~ "Failure"
| summarize FailureCount = count() by Caller, bin(TimeGenerated, 15m)
| where FailureCount >= 3
```



Design observation

The rule runs every hour with a one-hour lookback, but the query itself only looks at the last 15 minutes. As a result, each run inspects only the final quarter of its window, and failures earlier in the hour would be missed. The query time filter and the rule schedule should be designed together. Suggested improvement (not yet deployed in this lab):

```
AzureActivity
| where ActivityStatusValue =~ "Failure"
| summarize FailureCount = count(),
            Operations = make_set(OperationNameValue, 10),
            ResourceGroups = make_set(ResourceGroup, 10)
            by Caller, bin(TimeGenerated, 15m)
| where FailureCount >= 3
```

8. Alert or incident review

When the rule fires, the review routine I practised is:

1. Open the incident or alert and note the timestamp and the 15-minute window.
2. Identify the caller and whether it is an expected user or service principal.
3. List the operations and resource groups involved and whether they are related.
4. Check the surrounding Activity log for context: what else did this caller do before and after?
5. Classify the outcome (benign, needs follow-up, or suspicious) and document the reasoning.

Outcome in this lab: the environment is a single-user lab, so there were no matching bursts of failed operations during the test window, and no incident from this rule appears in the captured evidence. Documenting a “no alert” result and the reason for it is itself part of the exercise.

Author note: *if an incident did fire, replace this paragraph with your review of the timestamp, caller, operation and context.*

9. Findings and learning

What worked

- Standing up the full chain: resource group → Log Analytics workspace → Sentinel → Azure Activity solution → custom scheduled rule.
- Writing KQL that projects the fields needed for triage: time, caller, operation, resource group and status.
- Expressing the detection as a simple, explainable aggregation: failures per caller per 15-minute bin, filtered at a threshold.

What I learned

- **Validate the pipeline before trusting a detection.** My first validation query returned no results. A rule built on an empty table cannot fire, so confirming ingestion comes first.
- **Rule schedule and query window must line up.** A one-hour schedule with a 15-minute query filter leaves gaps in coverage.

- **Failure is not the same as malicious.** Most Activity-log failures are operational noise; useful detections need context and tuning.
- **Portal Activity log and workspace table are different views.** Data reaches the AzureActivity table only once the connector is working.

Important: this is a learning lab, not a production SOC deployment.

10. Tools and technologies

Tool	Use in this project
Microsoft Azure	Subscription, resource group and cloud platform
Microsoft Sentinel	SIEM: data connector, analytics rule, incident workflow
Log Analytics workspace	Storage and querying of collected logs
Azure Activity logs	Source of subscription control-plane events
KQL	Querying, aggregation and detection logic

11. Project status and limitations

Project status: In progress. The core build is complete (workspace, Sentinel, Azure Activity solution, enabled analytics rule). Re-testing log validation and testing the rule with deliberately generated failures are still to do.

Limitations

- Covers Azure Activity logs only. It does not, by itself, prove Windows endpoint monitoring, malware detection, identity sign-in analysis or detection of real attacks. Those would require additional data sources and testing.
- The detection rule has not been validated with deliberately generated test events, so its alerting behaviour is unproven.
- No entity mapping, MITRE ATT&CK mapping or automated response is configured.
- Single-user, single-subscription lab with no realistic background traffic.

Possible improvements

- Fix the query window as described in Section 7.
- Generate three or more failing operations from one account within 15 minutes (for example, repeated deployments of an intentionally invalid template) and document the alert end to end.
- Add entity mapping for the account and enrich the alert with operation and resource-group lists.
- Add more data sources, such as Microsoft Entra ID sign-in logs, and build cross-source detections.
- Create a workbook to visualise failures over time and try an automation rule or playbook.

Cost and clean-up

The lab used an Azure free-credit subscription. Microsoft Sentinel has offered a free trial for new workspaces, but terms change, so check current pricing and set a budget alert. When

finished, delete the resource group rg-sentinel-cloudsoc to remove the workspace and all Sentinel data.

12. LinkedIn project description

Microsoft Sentinel Cloud SOC Lab (Personal Learning Project)

- Configured a Microsoft Sentinel workspace and Azure Activity log collection.
- Used KQL to review Azure activity and investigate failed operations.
- Created a scheduled analytics rule for repeated failed Azure operations.
- Documented the build, findings and limitations, including how to improve rule design and test detections.

Tools: Microsoft Azure, Microsoft Sentinel, Log Analytics, KQL.

Note: keep only the bullets that accurately reflect work completed.